



Digital Design Verification

Lab Manual # 18 – RISC-V Datapath Implementation (I-Type and R-Type Instructions)

Release: 1.0

Date: 10-Sept-2024

NUST Chip Design Centre (NCDC), Islamabad, Pakistan



Copyrights ©, NUST Chip Design Centre (NCDC). All Rights Reserved. This document is prepared by NCDC and is for intended recipients only. It is not allowed to copy, modify, distribute or share, in part or full, without the consent of NCDC officials.

Revision History

Revision Number	Revision Date	Revision By	Nature of Revision	Approved By
1.0	10/06/2024	Ali Aqdas	Complete manual	-
2.0	18/8/2025	Umer Farooq / Fawad Ahmed	Manual Update	-



Contents

Contents	2
Objective.....	3
Tools	3
Instructions	3



Objective

The objective of this lab is to

- Design an accurate RISC-V Datapath for I & R-Type Instructions

Tools

- Cadence Xcelium

Instructions

The students are required to design and submit the following files for evaluation of this lab

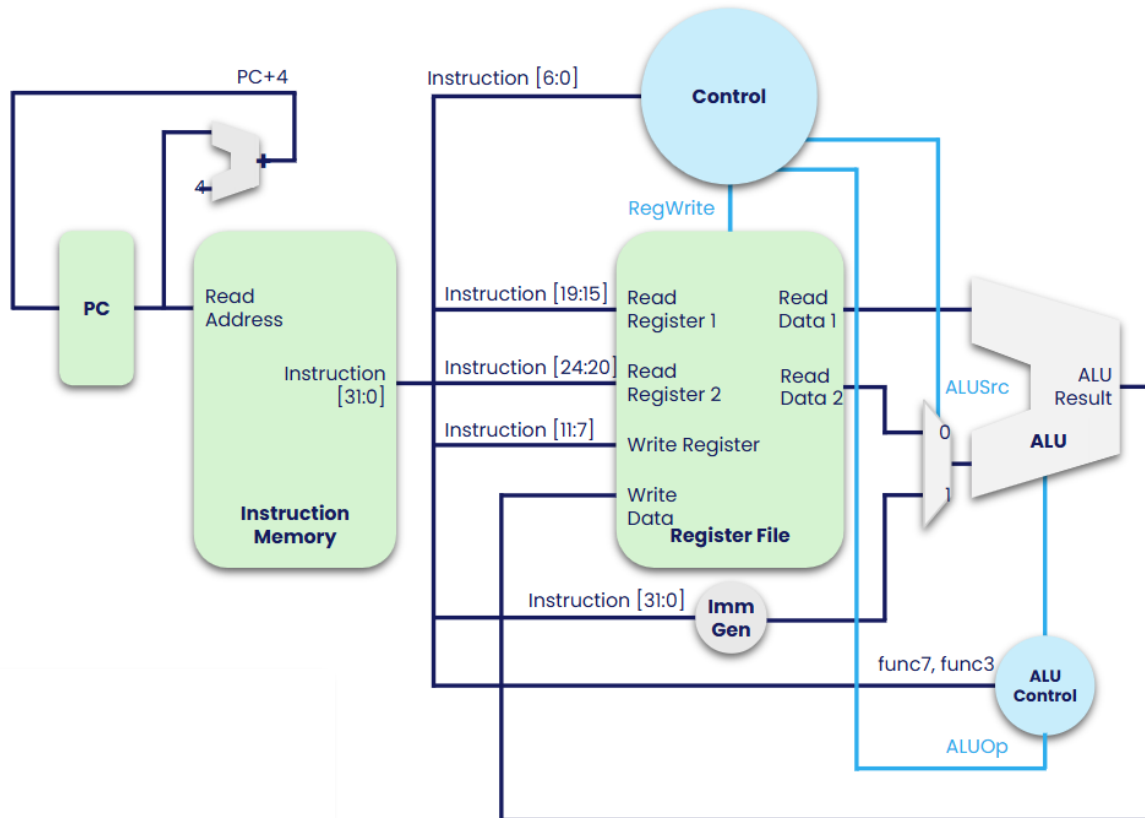
./<student-name>/	./rtl/	program_counter.sv inst_mem.sv reg_file.sv imm_gen.sv alu_logic.sv top.sv
	./tb/	program_counter_tb.sv inst_mem_tb.sv reg_file_tb.sv imm_gen_tb.sv alu_logic_tb.sv top_tb.sv



RISC-V Datapath

The microarchitecture of the RISC-V data-path comprised of the following components

- Program Counter
- Instruction Memory
- Data Memory
- Register File
- Arithmetic Logic Unit
- Immediate Generation Unit
- Control Logic



The datapath follows the same general stage of computation:

1. Instruction Fetch (IF): Send Program Counter to Instruction Memory and retrieve the instruction to be run.
2. Instruction Decode (ID): Read the instruction, decode what the instruction must do, compute immediate/register values
3. Execute (EX): Do any computations needed for the instruction
4. Memory (MEM): For data memory operations a separate stage for accessing Memory
5. Write Back (WB): Update the inputs to any state elements that must be updated by waiting for the next rising edge to write back the data.

A brief description of each of these components is available in the text below.



Program Counter

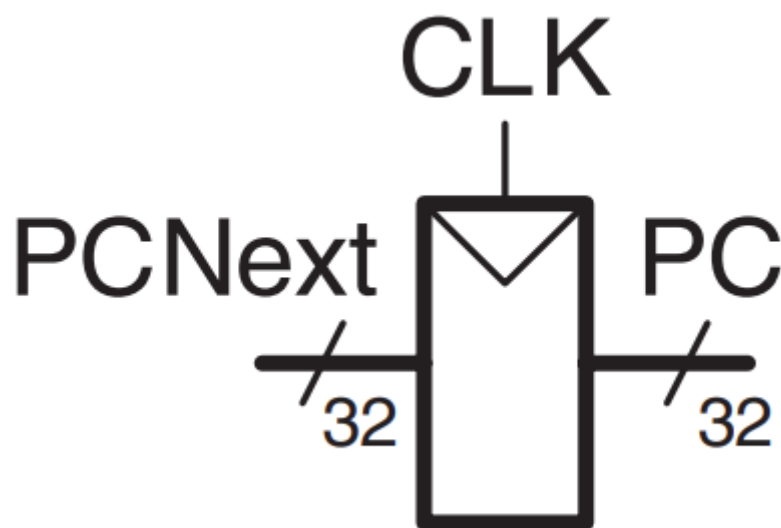
The Program Counter (PC) is a special register in the RISC-V processor that holds the address of the instruction currently being executed (or the next instruction to be fetched).

It is 32-bit wide in RV32 and 64-bit wide in RV64 architectures.

Role of the PC

1. At the start of execution, the PC is initialized to the reset vector (usually address 0x00000000 or implementation-defined).
2. The instruction at the PC address is **fetched** from memory.
3. After fetching, the PC is updated:
 - Normally: $PC = PC + 4$ (since RISC-V instructions are 32 bits = 4 bytes).
 - For branches/jumps: $PC = \text{Target Address}$ (calculated by ALU or branch unit).

Thus, the PC is the **heart of instruction flow control**.





Register File

The register file is a **state element** and has the following behavior

Inputs			Outputs		
Function	Size	Port Name	Function	Size	Port Name
Data Input Bus	32-Bit	dataW	Data Output	32-Bit	data1 data2
Selectors	5-Bit	rs1 rs2 rs3			
Write Enable (1=Enable, 0 = Disable)	1-Bit	RegWEn			

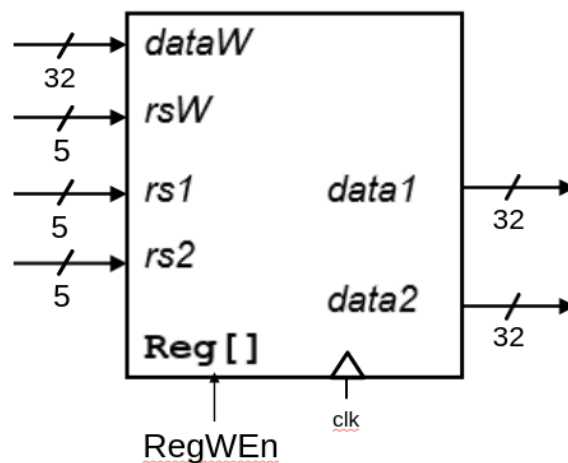
The selector **rs1** identifies which register should output the data in **data1**.

The selector **rs2** identifies which register should output the data in **data2**

The selector **rsW** identifies which register should update next cycle.

The clock behavior of the register file must be as follows.

1. If **RegWEn** is **1** on rising clock edge, set the register corresponding to **rsW** to **dataW**
2. Reading the value of register doesn't require a clock tick. Only writing to the register takes a tick.
3. Data Computed in the Cycle N will get written on the **rising edge** of the next cycle





Main Memory

Memory is also a **state element** that holds both instructions and data in one contiguous 32-bit memory space. The behavior of

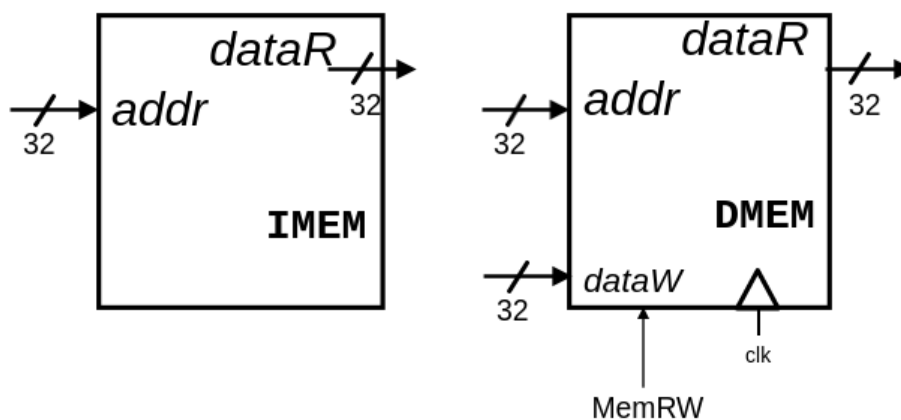
Inputs			Outputs		
Function	Size	Port Name	Function	Size	Port Name
Input Address	32-Bit	addr	Data Output	32-Bit	dataR
Input Data	32-Bit	dataW			
Write Enable (1=Enable, 0 = Disable)	1-Bit	MemRW			

Reading a memory location does not require clock tick to trigger. However, a clock trigger causes the memory to write the data of the previous cycle

Generally, memory is partitioned into two parts, one for the instructions and one for the data, however for simplicity, we are going to create two memories.

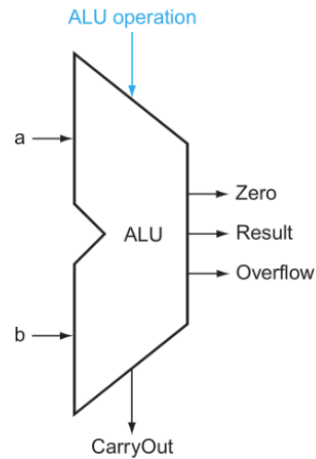
1. **Instruction Memory:** A read only memory for fetching instructions. Since it's a read-only memory it never requires a clock trigger.
2. **Data Memory:** A memory for loading (reads) and storing (writes) of data words.

Under the hood, these are just placeholders for caches (more on this later).



Arithmetic Logic Unit

In each cycle, every logic block/state element needs to do something, and all blocks run simultaneously. The general philosophy of a CPU operation relies on performing every single operation for each instruction even if the result doesn't make sense and then use a control logic to ensure that only the operations we want, end up affecting the state elements. This can be done through a multiplexer which can receive multiple inputs (e.g. $a+b$, $a \mid b$, $a - b$) and a selector (control signal from the control logic) and generate an output based on the corresponding inputs.



Control Logic

It consists of two control logics:

- **ALU Control**

ALU control lines	Function
0000	AND
0001	OR
0010	add
0110	subtract

ALU is used for:

- Arithmetic-logic operations in R-type instructions!
- Address calculation (base + offset) in loads/stores
- Comparing operands in branch instructions
- Subtract and check zero result signal to test for equality!

Input

- funt7 and funct3 bits
- 2-bit control signal ALUOp
 - ALUOp is generated by main control unit using **OpCode Bits**
 - Such implementation simplifies main control unit!
 - ALUOp Value
 - 00: Load / Store
 - 01: beq
 - 10: R-type Instruction (add, sub, and, or)
 - ALUOp is generated based on type of instruction!

Output

- ALU Control Lines

The ALU control signals are given below:



Instruction opcode	ALUOp	Operation	Funct7 field	Funct3 field	Desired ALU action	ALU control input
lw	00	load word	XXXXXXX	XXX	add	0010
sw	00	store word	XXXXXXX	XXX	add	0010
beq	01	branch if equal	XXXXXXX	XXX	subtract	0110
R-type	10	add	0000000	000	add	0010
R-type	10	sub	0100000	000	subtract	0110
R-type	10	and	0000000	111	AND	0000
R-type	10	or	0000000	110	OR	0001

Truth Table

ALUOp		Funct7 field							Funct3 field			Operation
ALUOp1	ALUOp0	I[31]	I[30]	I[29]	I[28]	I[27]	I[26]	I[25]	I[14]	I[13]	I[12]	
0	0	X	X	X	X	X	X	X	X	X	X	0010
X	1	X	X	X	X	X	X	X	X	X	X	0110
1	X	0	0	0	0	0	0	0	0	0	0	0010
1	X	0	1	0	0	0	0	0	0	0	0	0110
1	X	0	0	0	0	0	0	0	1	1	1	0000
1	X	0	0	0	0	0	0	0	1	1	0	0001

- Main Control

The input for main control is opcode only and the output signal is given below:

Instruction	ALUSrc	Memto-Reg	Reg-Write	Mem-Read	Mem-Write	Branch	ALUOp1	ALUOp0
R-format	0	0	1	0	0	0	1	0
lw	1	1	1	1	0	0	0	0
sw	1	X	0	0	1	0	0	0
beq	0	X	0	0	0	1	0	1

Truth Table

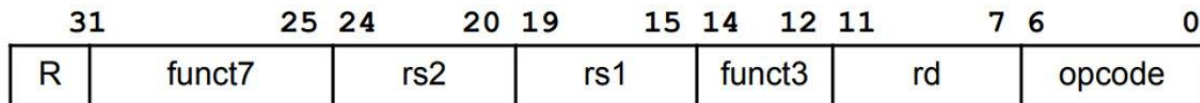
Input or output	Signal name	R-format	lw	sw	beq
Inputs	I[6]	0	0	0	1
	I[5]	1	0	1	1
	I[4]	1	0	0	0
	I[3]	0	0	0	0
	I[2]	0	0	0	0
	I[1]	1	1	1	1
	I[0]	1	1	1	1
Outputs	ALUSrc	0	1	1	0
	MemtoReg	0	1	X	X
	RegWrite	1	1	0	0
	MemRead	0	1	0	0
	MemWrite	0	0	1	0
	Branch	0	0	0	1
	ALUOp1	1	0	0	0
	ALUOp0	0	0	0	1



Instructions Formats

R-Type Instructions

The R-Type Instructions have the following encoding



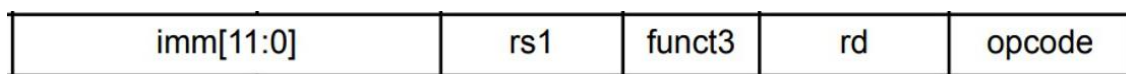
and include the instructions given in the table below

op	funct3	funct7	Type	Instruction	Description	Operation
0110011 (51)	000	0000000	R	add rd, rs1, rs2	add	rd = rs1 + rs2
0110011 (51)	000	0100000	R	sub rd, rs1, rs2	sub	rd = rs1 - rs2
0110011 (51)	001	0000000	R	sll rd, rs1, rs2	shift left logical	rd = rs1 << rs2 _{4:0}
0110011 (51)	010	0000000	R	slt rd, rs1, rs2	set less than	rd = (rs1 < rs2)
0110011 (51)	011	0000000	R	sltu rd, rs1, rs2	set less than unsigned	rd = (rs1 < rs2)
0110011 (51)	100	0000000	R	xor rd, rs1, rs2	xor	rd = rs1 ^ rs2
0110011 (51)	101	0000000	R	srl rd, rs1, rs2	shift right logical	rd = rs1 >> rs2 _{4:0}
0110011 (51)	101	0100000	R	sra rd, rs1, rs2	shift right arithmetic	rd = rs1 >>> rs2 _{4:0}
0110011 (51)	110	0000000	R	or rd, rs1, rs2	or	rd = rs1 rs2
0110011 (51)	111	0000000	R	and rd, rs1, rs2	and	rd = rs1 & rs2

In this type, both the operands are from register file, while the instruction specifies the register names to be used in **rs1** and **rs2**

I-Type Instructions

These instructions have one immediate operand which is specified in the instruction encoding as can be seen from the encoding below

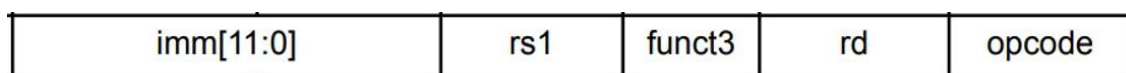


A list of immediate instructions is shown in table below

op	funct3	funct7	Type	Instruction	Description	Operation
0010011 (19)	000	-	I	addi rd, rs1, imm	add immediate	rd = rs1 + SignExt(imm)
0010011 (19)	001	0000000*	I	slli rd, rs1, uimm	shift left logical immediate	rd = rs1 << uimm
0010011 (19)	010	-	I	slti rd, rs1, imm	set less than immediate	rd = (rs1 < SignExt(imm))
0010011 (19)	011	-	I	sltiu rd, rs1, imm	set less than imm. unsigned	rd = (rs1 < SignExt(imm))
0010011 (19)	100	-	I	xori rd, rs1, imm	xor immediate	rd = rs1 ^ SignExt(imm)
0010011 (19)	101	0000000*	I	srlu rd, rs1, uimm	shift right logical immediate	rd = rs1 >> uimm
0010011 (19)	101	0100000*	I	srai rd, rs1, uimm	shift right arithmetic imm.	rd = rs1 >>> uimm
0010011 (19)	110	-	I	ori rd, rs1, imm	or immediate	rd = rs1 SignExt(imm)
0010011 (19)	111	-	I	andi rd, rs1, imm	and immediate	rd = rs1 & SignExt(imm)

Immediate Generation Unit

31 30 29 28 27 26 25 24 23 22 21 20 19 18 17 16 15 14 13 12 11 10 09 08 07 06 05 04 03 02 01 00



The least significant 12 bits of generated immediate value are directly extracted from the instruction, and the remaining bits are sign extended. Hence, bit 32 would be the sign bit of the immediate.



Lab Task

Again, revisiting the system level diagram, you are required to design the data-path shown in the diagram below. The datapath with control logic must support all I-Type and R-Type Instructions.

